Project Report - Addis Ababa Bus Tracking System

Abstract

People who ride public buses in Addis Ababa mostly cannot tell where their bus is or when it will arrive. This project builds a real-time bus tracking system that needs no hardware in the bus. The crew's phone sends GPS points to a backend, which matches them to the road, estimates arrival times as honest ranges, and follows buses through terminals. Riders use a mobile web app with a live map, a trip planner and stop pages. Drivers use a simple app with crowding and terminal buttons, and an operator uses an admin app to manage drivers, vehicles and shifts. The system was built with Node.js, PostgreSQL with PostGIS and React, and was tested with a simulated bus on routes 49 and 61. It is deployed online on free hosting. A real test ride is the next step.

1. Introduction

Background. In Addis Ababa, people who ride public buses mostly cannot tell where their bus is or how long they will wait. They stand at a stop and guess. Almost every phone carries GPS and a browser, so the position of a bus can be shared at no hardware cost.

Problem. A rider needs three answers: which bus goes where I am going, where is it now, and when will it reach my stop. Raw GPS dots do not give these. The positions must be matched to the road, turned into arrival times, and shown honestly when the signal is weak.

Objectives.

1. Let the bus crew share a bus's position from a phone with very little effort.
2. Show riders live buses and arrival ranges on a map, and plan a trip from one place to another.
3. Show riders what happens at a terminal: which bus is boarding and when the next leaves.
4. Let an operator manage drivers, vehicles and shifts and watch the fleet.
5. Never show a precise time the data cannot support.

Scope. Two routes, 49 and 61, each in both directions, with three web apps (rider, driver, admin), one backend and one database. Tickets, payment and fares are outside the scope.

2. Existing approaches and gap

Riders today mostly rely on three things: asking other people, waiting and watching. This project needs no cooperation from an operator's systems and no device in the bus. The crew's own phone sends the

position, and the system works out the rest. It also handles what a fixed timetable cannot: buses that wait at a terminal until they are full, buses that stand in traffic, and buses that go quiet.

3. Methodology and system overview

The system has four parts. The driver app sends GPS points. The backend turns them into positions, arrival times and departures. The rider app reads the results. The admin app manages drivers, vehicles and shifts. All data lives in one PostgreSQL database with the PostGIS map extension. The System Design document has the architecture and database diagrams.

The work followed a build-and-test cycle:

1. **Backend first.** Each feature has four files: routes, controller, service and repository. The rules that decide things, such as the arrival range or whether a bus is waiting, are pure functions in their own files, so each has a test script.
2. **Placeholders for unknown numbers.** Limits that need real measurements, such as the longest wait at a terminal, sit in one file as marked estimates, so they can be replaced after a real test ride.
3. **A fake bus.** A script drives a bus along a real route and sends GPS points like a phone would. It can stop, wait, go quiet and drive wrong, so every case can be tested at a desk.
4. **Apps in order.** Driver app, rider app and admin app were built on the finished backend, and each was tested against the fake bus.
5. **Deployment last.** The same code was then put online on free hosting and checked again.

4. Implementation

Technology

Part	Technology
Backend	Node.js, Express 5, TypeScript, Drizzle ORM
Database	PostgreSQL with PostGIS (Docker locally, Neon online)
Rider, driver and admin apps	React with TypeScript and Vite. Tailwind CSS in the rider app
Map	Leaflet with OpenStreetMap tiles

Hosting	Render (backend), Vercel (apps), Neon (database)

From GPS to an estimate. The driver app sends batches of points every 5 seconds, each with its own time, and queues them when the phone is offline. The backend matches each point to the road line of the bus's route direction, which gives its progress from 0 to 1, and averages the speed over the last 45 seconds. The arrival time for a stop is the sum of the stretch times between the bus and the stop. Stretch times start as guesses and are replaced each night by the median of real crossings. The answer is a range, and it is withheld when the newest point is older than 90 seconds.

Terminals. A bus within 100 m of a terminal is waiting there. The rider sees the waiting buses, the one that is boarding, and the minutes since the last bus left. A bus counts as departing soon when it is full, when the crew taps Departing now, or after the usual 10minute layover.

Trips. The rider picks two places. The backend finds the stops at each place and the route directions that visit them in the right order, so the rider never picks a direction. For each result it adds where to wait and the next bus.

Warnings and shifts. The backend detects wrong way, off the road, a wrong phone clock and no signal. It hides wrong-way and off-road buses from riders. A shift with no point for 30 minutes ends by itself.

The rider screen. One map stays alive behind two tabs. Results slide up as a sheet with three heights, and tapping a route card highlights its line and bus. The search and chosen route are kept in the page address.

5. Deployment

The system runs online on three free services:

Part	Service
Database	Neon, PostgreSQL with PostGIS
Backend	Render, a Docker service that applies the migrations and loads the route data on every start
Rider, driver and admin apps	Vercel, one project for each app

6. Limitations and future work

Limitations

- Everything was tested with a simulated bus. No real bus has carried the system yet, so the limits (for example the longest wait at a terminal) are still estimates.
- A web app can stop sending GPS when the phone locks or the browser goes to the background. The crew must keep the driver app open.
- Estimates are rough at first. They improve only as real trips teach the system the stretch times.
- Two routes are loaded. The interface is English only, though stop names are stored in Amharic.

Future work

1. A real test ride on routes 49 and 61, then replace the estimated limits with measured ones.
2. A native driver app that keeps sending GPS in the background.
3. An Amharic interface with a language switch.
4. More routes, loaded by the admin app instead of by a script.
5. Notifications such as "your bus is two stops away".

7. Conclusion

The project shows that a bus crew's phone is enough to give riders a live map and honest arrival times. The system covers the whole chain: GPS from the driver, position on the road, arrival ranges, terminal departures, trip planning, an operator view and online hosting. Its main design choice is honesty: when data is old or missing, it says so instead of guessing. The next step is a real test ride, which will replace the estimated limits with measured ones.